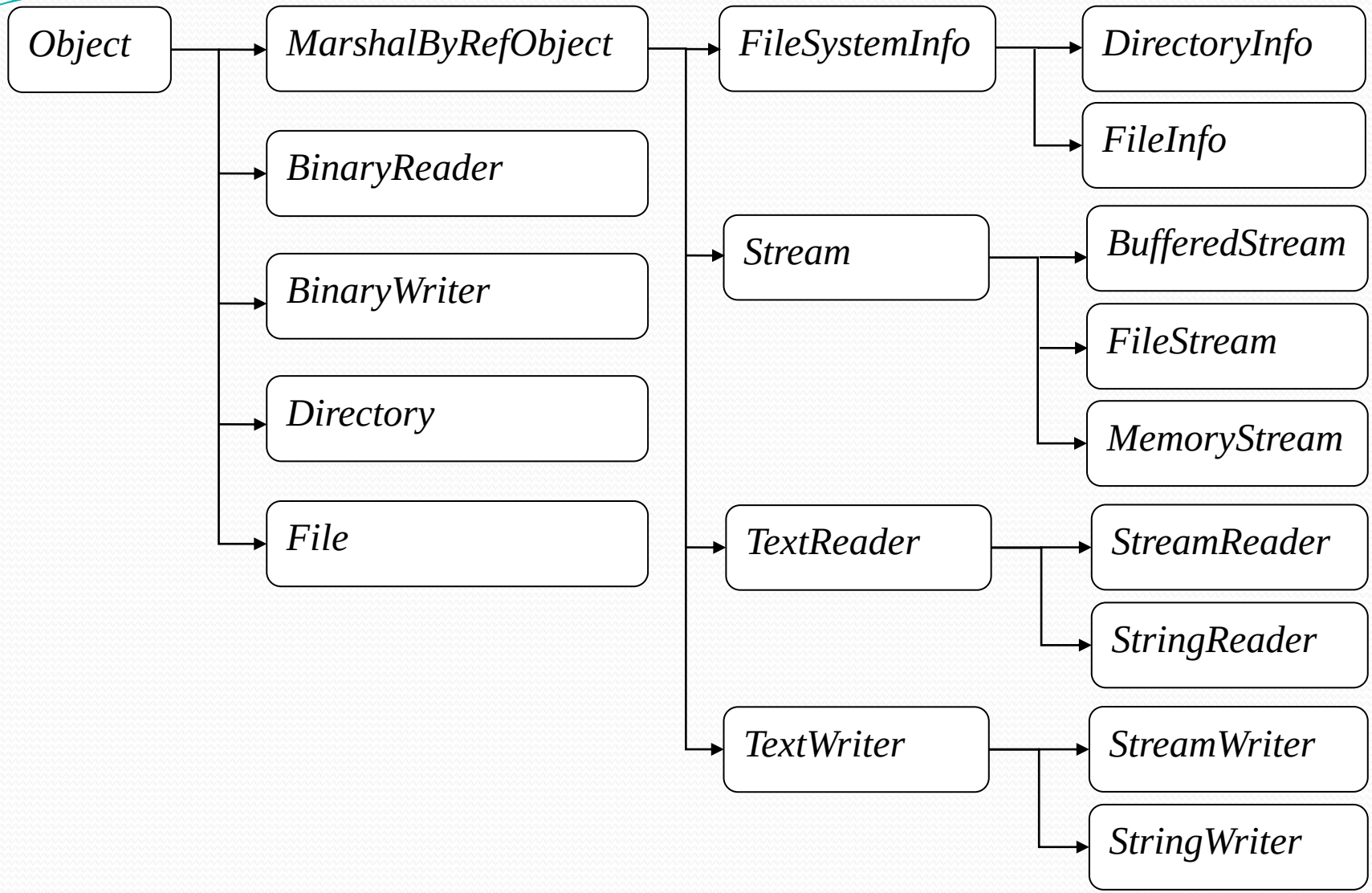
Основы работы с файлами в C#

Основы работы с файлами в C#

- Работа с файлами в языке *C#* реализуется посредством потоков
- Поток – абстрактное понятие, описывающее любой перенос данных от источника к приемнику (именно потоки позволяют считывать данные из файла и записывать данные в файл)
- Поток представляет собой последовательность байтов и не зависит от конкретного устройства, с которым производится обмен (оперативная память, файл на диске, клавиатура, принтер)

Основы работы с файлами в C#

- Для поддержки потоков библиотека *.NET* (*FCL*) содержит иерархию классов



Иерархия классов в пространстве имен ***System.IO***

Классы *TextReader* и *StreamReader*

Классы *TextReader* и *StreamReader*

- позволяют преобразовывать байтовые потоки в символьные и наоборот

- класс *StreamReader* является потомком абстрактного базового класса *TextReader*, предоставляющего классам-наследникам набор методов, реализующих считывание последовательности символов или строк из потока (файла)

Некоторые методы класса *TextReader*

Название	Описание
***Peek*()**	Возвращает следующий символ из потока, при этом указатель текущей позиции не перемещается (удобно использовать для проверки наличия следующего символа в потоке: если конец файла достигнут, возвращаемое значение равно –1)
***Read*()**	Первый вариант: считывает текущий символ из потока и передвигает указатель на один символ в соответствии с кодировкой. Если доступных для считывания символов нет, возвращает значение –1. Второй вариант: считывает из потока последовательность символов и записывает ее в массив символов, начиная с определенного индекса. Метод возвращает фактическое количество считанных символов, либо нуль при отсутствии символов, доступных для чтения
***ReadBlock*()**	Блокирующая версия метода *Read*()
***ReadLine*()**	Считывает строку символов из текущего потока и возвращает данные в строковом формате либо значение *null*, если достигнут конец потока входных данных
***ReadToEnd*()**	Считывает все символы, начиная с текущей позиции, до конца потока как одну строку

Разница между блокирующими и неблокирующими методами чтения

- Обмен с потоком для повышения скорости передачи данных производится, как правило, через специальную область оперативной памяти – буфер (он выделяется для каждого открытого файла)
- При чтении из файла данные вначале считываются в буфер (не столько, сколько запрашивается, а сколько помещается в буфер), а уже из буфера передаются в поток
- В процессе считывания возможна ситуация, когда буфер пуст, т. е. данные пока не доступны, но ожидаемы в будущем. В такой ситуации
  - блокирующие методы блокируют процесс считывания, помещая его в режим сна до тех пор, пока не появятся доступные для чтения данные
  - неблокирующие методы просто возвращают в процесс некоторое значение, указывающее на то, что доступных для чтения данных нет

- Примеры 1-4

- ```
  string[] Row = InputFile.ReadLine().Split(div,
  StringSplitOptions.RemoveEmptyEntries);
  ```
  ```

Классы *TextWriter* и *StreamWriter*

Класс *TextWriter*

- Абстрактный класс *TextWriter* содержит методы, позволяющие производить запись символов в байтовый поток (файл)

Методы класса *TextWriter*

Название	Описание
***Close*()**	Закрывает файл, освобождает связанные с ним ресурсы
***Write*()**	Осуществляет запись данных в текстовый поток. Метод позволяет принимать в качестве параметра данные, представленные в разных форматах (целые или вещественные числа, символы или массивы символов, строки, логические переменные)
***WriteLine*()**	Записывает в текстовый поток данные, за которыми следует признак конца строки

Класс *StreamWriter*

- Класс *StreamWriter* является классом-наследником класса *TextWriter* и реализует методы класса-предка
- Конструктор StreamWriter(<ИмяФайла>) создает текстовый поток для записи данных в заданный файл
- Если файл существует, он будет перезаписан, в противном случае создается новый файл
- Пример 5

Классы *Stream* и *FileStream*

Класс *Stream*

- является абстрактным базовым классом всех потоков
- При работе с потоком допустимы три основные операции:
  - чтение из потока – перенос информации из потока в структуру данных, такую, как массив байтов
  - запись в поток – передача данных из структуры данных в поток
  - поиск, включающий в себя отправку запросов и изменение текущей позиции внутри потока

Свойства и методы класса *Stream*

Название	Вид	Описание
CanRead	Неизменяемое свойство	Показывает, можно ли ввести данные из потока
CanWrite	Неизменяемое свойство	Показывает, можно ли записать данные в поток
CanSeek	Неизменяемое свойство	Показывает, поддерживает ли поток произвольный доступ к данным, при котором можно задать текущее положение в потоке
Close()	Экземплярный метод	Закрывает текущий поток и освобождает все связанные с ним ресурсы
Flush()	Экземплярный метод	Записывает данные из буфера в связанный с потоком источник данных (файл) и очищает буфер

Свойства и методы класса *Stream*

Название	Вид	Описание
Length	Неизменяемое свойство	Возвращает длину потока в байтах
Position	Изменяемое свойство	Возвращает либо задает текущее положение в потоке
Read()	Экземплярный метод	Считывает последовательность байтов определенной длины из потока и записывает их в одномерный массив байт. Возвращает количество прочитанных байтов либо ноль, если достигнут конец потока
ReadByte()	Экземплярный метод	Возвращает целочисленное представление следующего байта из потока либо –1, если достигнут конец потока

Свойства и методы класса *Stream*

Название	Вид	Описание
ReadTimeOut	Изменяемое свойство	Возвращает или задает продолжительность времени ожидания в миллисекундах в процессе чтения данных из потока
Seek()	Экземплярный метод	Перемещает указатель текущего положения в потоке на заданную позицию
Write()	Экземплярный метод	Записывает последовательность байт определенной длины в массив, начиная с указанного индекса, и перемещает указатель в потоке на количество записанных байт
WriteByte()	Экземплярный метод	Записывает один байт в поток и перемещает указатель в потоке на один байт
WriteTimeOut	Изменяемое свойство	Возвращает или задает продолжительность времени ожидания в миллисекундах в процессе записи данных в поток

Класс *FileStream*

- Ввод/вывод, организованный на уровне байтов, называется байтовым потоком
- Для создания байтового потока служит класс *FileStream* - класс-наследник класса *Stream,* реализующий рассмотренные выше методы и свойства
- Конструктор класса перегружен и имеет несколько вариантов реализации
  - FileStream (<Имяфайла>,<Режим>,<ТипДоступа>)

  создает экземпляр класса *FileStream*, связывает его с заданным файлом и открывает файл в заданном режиме с заданным типом доступа

Режимы открытия файла

- Задается одним из константных значений перечисления *FileMode*:
  - *Append* – указывает, что операционная система (ОС) должна открыть существующий файл для записи, при этом текущий указатель положения в файле устанавливается в конец файла. Если файл не существует, создает новый файл
  - *Create* – указывает, что ОС должна создать новый файл. Если в каталоге уже существует файл с заданным именем, он будет переписан
  - *CreateNew* – указывает, что ОС должна создать новый файл. Если в каталоге уже существует файл с заданным именем, генерируется исключение *IOException*
  - *Open* – указывает, что ОС должна открыть существующий файл. Если файл с заданным именем не существует, генерируется исключение *FileNotFoudException*
  - *OpenOrCreate* – указывает, что ОС должна открыть файл с заданным именем, если он существует, либо создать файл с заданным именем в противном случае
  - *Truncate* – указывает, что ОС должна открыть существующий файл. После открытия он должен быть обрезан до нулевой длины. Файл может быть открыт только для записи

Типы доступа к файлу

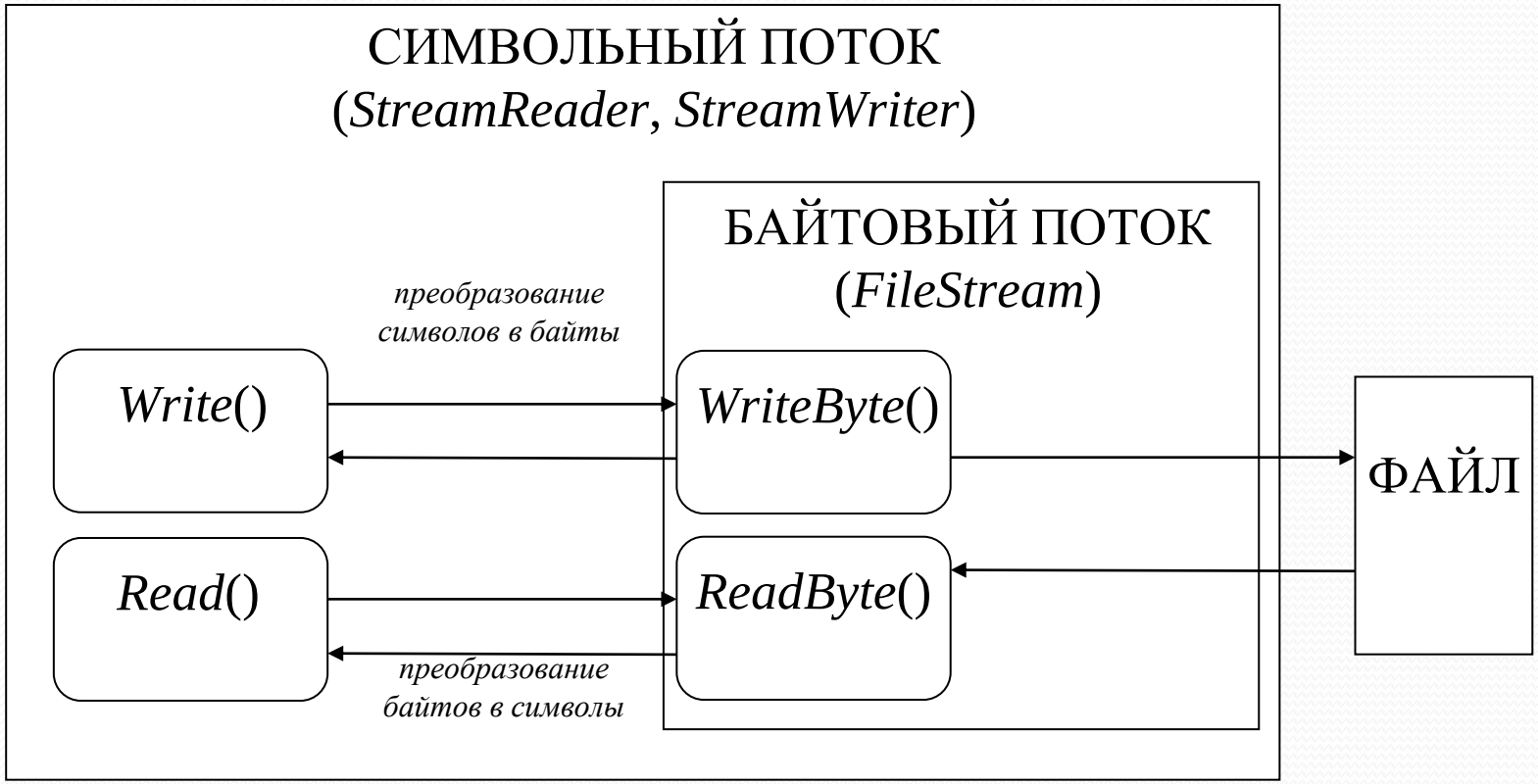
- Принимает одно из константных значений перечисления *FileAccess*:
  - *Read* – указывает, что файл открыт только для чтения
  - *Write* – указывает, что файл открыт только для записи
  - *ReadWrite* – указывает, что файл доступен и для чтения и для записи

Пример 6

Метод *Seek*()

- Используя метод *Seek*() для позиционирования указателя текущего положения в файле, можно управлять процессом считывания данных из файла
- Синтаксис: Seek(long offset, SeekOrigin origin)
  - параметр *offset* задает смещение в байтах относительно точки отсчета положения в потоке
  - точка отсчета задается параметром *origin* и может принимать одно из константных значений перечисления *SeekOrigin*:
    - *Begin* – начало файла
    - *Current* – текущее положение
    - *End* – конец файла
- Пример 7

- Байтовый поток удобно использовать только для решения задач, в которых операции с файлами реализуются программным путем и пользователю не требуется самому изменять содержимое файлов
- Если необходимо считать из файла данные, введенные пользователем и представленные в символьном формате, удобнее пользоваться классами *StreamReader* и *StreamWriter* (надстройки над классом *Stream* - тоже используют байтовые потоки для чтения данных из файла и записи данных в файл, но при этом выполняют автоматическое преобразование байтового потока в символьный и наоборот)


СИМВОЛЬНЫЙ ПОТОК
(StreamReader, StreamWriter)
БАЙТОВЫЙ ПОТОК
(FileStream)
преобразование
символов в байты
Write()
WriteByte()
ФАЙЛ
Read()
ReadByte()
преобразование
байтов в символы

Классы *BinaryReader* и *BinaryWriter*

Двоичные потоки

- Кроме байтовых и символьных потоков в языке *C#* реализованы двоичные потоки для работы с примитивными типами данных (*int*, *double*, *short* и др.)
- Двоичные потоки хранят значения в двоичном коде и не предназначены для просмотра их человеком
- Двоичный поток открывается на основе базового потока, который, в частности, может быть инициализирован с помощью класса *FileStream*
- Конструктор BinaryReader(<ИмяБазовогоПотока>) открывает двоичный поток для считывания данных из файла, ассоциированного с заданным базовым потоком

Свойства и методы класса *BinaryReader*

Название	Вид	Описание
BaseStream	Неизменяемое свойство	Предоставляет доступ к базовому потоку
Close()	Экземплярный метод	Закрывает текущий и связанный с ним базовый поток
PeekChar()	Экземплярный метод	Возвращает следующий доступный для чтения символ в формате int, не перемещая внутренний указатель текущего положения в потоке. Если доступного для чтения символа нет, возвращает значение –1
Read()	Экземплярный метод	Считывает текущий символ из базового потока и перемещает внутренний указатель потока на один символ либо возвращает значение –1, если доступных для чтения символов нет

Класс *BinaryReader*

- Метод *Read*() перегружен и имеет несколько вариантов реализации
  - его можно использовать для считывания из базового потока последовательности байтов определенной длины и записи их в одномерный массив с указанного индекса; метод при этом возвращает количество считанных из потока байт либо ноль, если достигнут конец потока
  - может считывать из базового потока последовательность символов
- Кроме того, в классе *BinaryReader* определены методы, реализующие считывание из базового потока значений всех встроенных в *C#* примитивных типов. Синтаксис таких методов одинаков и имеет вид: *ReadXX*(), где вместо *XX* указывается соответствующий тип данных (например, *ReadBoolean*() считывает из базового потока величину типа *bool* и перемещает внутренний указатель потока на соответствующее количество байт вперед в зависимости от типа считанной величины - на один байт)

Класс *BinaryWriter*

- Конструктор BinaryWriter(<ИмяБазовогоПотока>) открывает двоичный поток для записи данных в файл, ассоциированный с заданным базовым потоком

Методы класса *BinaryWriter*

Название	Описание
Flush()	Записывает данные из буфера в связанный с потоком источник данных (файл) и очищает буфер
Seek()	Устанавливает позицию указателя в текущем потоке (имеет тот же синтаксис, что и аналогичный метод класса Stream)
Write()	Записывает заданное значение в текущий поток
Close()	Закрывает текущий и связанный с ним базовый поток

- Пример 8

Классы *FileInfo* и *DirectoryInfo*

Классы *FileInfo* и *DirectoryInfo*

- Методы работы с файлами не ограничиваются только манипуляциями с данными в процессе ввода/вывода, а включают также такие операции, как копирование, перемещение, удаление, вывод информации об объекте файловой системы – файле или каталоге. Для реализации этих операций в языке C# предусмотрены следующие классы:
  - статический класс *Directory* и экземплярный класс *DirectoryInfo* позволяют создавать, копировать, перемещать, переименовывать и удалять каталоги, а также получать и задавать сведения о типе *DateTime*, относящиеся к созданию, доступу и записи каталога
  - статический класс *File* и экземплярный класс *FileInfo* позволяют создавать, копировать, удалять, перемещать и открывать файлы, а также могут использоваться при инициализации объектов *FileStream*

Свойства и методы класса *FileSystemInfo*

Название	Вид	Описание
Attributes	Изменяемое свойство	Возвращает или устанавливает атрибуты для данного объекта файловой системы. Использует константные значения перечисления *FileAttributes*
CreationTime	Изменяемое свойство	Получает или задает время создания объекта файловой системы
Delete()	Экземплярный метод	Удаляет текущий объект файловой системы – файл или каталог – без возможности его восстановления
Exists	Неизменяемое свойство	Определяет, существует ли данный объект файловой системы
Extension	Неизменяемое свойство	Возвращает расширение файла в строковом формате

Свойства и методы класса *FileSystemInfo*

Название	Вид	Описание
FullName	Неизменяемое свойство	Возвращает имя файла или каталога с указанием полного пути
LastAccessTime	Изменяемое свойство	Позволяет получить или задать время последнего обращения к объекту файловой системы
LastWriteTime	Изменяемое свойство	Позволяет получить или установить время последней операции записи в объект файловой системы
Name	Неизменяемое свойство	Для файлов возвращает имя файла, для каталогов – имя последнего каталога в иерархии, если таковая существует, а если нет, возвращает имя текущего каталога
Refresh()	Экземплярный метод	Обновляет состояние объекта. Используется перед попыткой получения сведений об атрибуте текущего объекта, иначе сведения могут оказаться устаревшими

- Описанные свойства и методы реализуются классами-наследниками *FileInfo* и *DirectoryInfo*
- Конструктор класса FileInfo(<ИмяФайла>) создает экземпляр класса и ассоциирует его с заданным файлом

Методы класса *FileInfo*

Название	Описание
CopyTo()	Первый вариант: если в параметрах указано только имя файла, то копирует текущий файл в новый файл, имя которого задано в качестве параметра. В случае, если создаваемый файл уже существует, его перезапись недопустима. Второй вариант: если кроме имени файла указан логический параметр, метод копирует текущий файл в новый файл и позволяет разрешить или запретить перезапись файла
Create()	Создает файл и инициализирует байтовый поток класса FileStream, связанный с заданным файлом
MoveTo()	Перемещает текущий файл в новое местоположение и разрешает переименование файла

Класс *DirectoryInfo*

- Конструктор DirectoryInfo(<Путь>) выполняет инициализацию нового экземпляра класса для заданного пути

- У класса *DirectoryInfo* есть свойство *Parent*, которое возвращает родительский каталог текущего подкаталога либо значение *null*, если путь к файлу пуст или указывает на корневой каталог

Методы класса *DirectoryInfo*

Название	Описание
Create()	Создает каталог, заданный в конструкторе класса. Если такой каталог уже существует, метод ничего не выполняет
CreateSubDirectory()	Создает один или несколько подкаталогов по заданному пути
GetDirectories()	Возвращает подкаталоги текущего каталога в виде массива объектов класса DirectoryInfo

Методы класса *DirectoryInfo*

Название	Описание
GetFiles()	Первый вариант: позволяет получить файлы в текущем каталоге в виде массива объектов класса FileInfo. Второй вариант: если указано имя файла, метод позволяет осуществить поиск всех файлов, имена которых удовлетворяют некоторому условию поиска, заданному в строковом формате, и возвращает массив объектов класса FileInfo либо null, если таких файлов нет. При указании в параметре имени файла разрешено применять стандартные подстановочные знаки ‘*’ и ‘?’, используемые при поиске
MoveTo()	Перемещает каталог и все его содержимое в местоположение, на которое указывает новый путь

- Пример 9